



Centre universitari adscrit a la



**Universitat
Pompeu Fabra**
Barcelona

**Bachelor in Computer Engineering — Management and
Information Systems Ethical Pentesting in Virtualized**

Environments with EVE-NG Annexos Volume II

Eloi Egea Rada Supervisor: Pere Vidiella i Catalan

April 2026

Contents

1	EVE-NG Installation on Proxmox VE	1
2	Lab 1: PEBCAK Corp — Teacher Reference	3
3	Lab 2: SYNAPSE Intelligence Portal — Teacher Reference	5
4	Lab 3: HELIX Systems — Teacher Reference	8
5	Cybersecurity Training Platform Descriptions	10
5.1	HackTheBox	10
5.2	TryHackMe	10
5.3	Cisco Networking Academy	11
5.4	SEED Labs	11
5.5	Game of Active Directory (GOAD)	11
5.6	VulnHub	11
5.7	Hack4u Academy	12
6	Objectives: Deliverables, Detailed KPIs, and Audience	13
6.1	Deliverables by Specific Objective	13
6.1.1	OE1: Design and Implement EVE-NG Labs	13
6.1.2	OE2: Develop Automation Scripts	13
6.1.3	OE3: Develop Structured Teaching Material	13
6.1.4	OE4: Validate Usability and Educational Effectiveness	14
6.2	Detailed KPI Tables	14
6.2.1	Technical Indicators	14
6.2.2	Educational Indicators	15
6.2.3	Quality Indicators	15
6.3	Extended Audience Analysis	15

6.3.1	Secondary End Users	15
6.3.2	Potential Audience Beyond Tecnocampus	16
7	Methodology: Detailed Phase Activities and Quality Assurance	17
7.1	Detailed Phase Activity Lists	17
7.1.1	Phase 0: Preparation and Infrastructure (July–November 2025)	17
7.1.2	Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026)	17
7.1.3	Phase 2: Implementation and Interim Validation (January–March 2026)	19
7.1.4	Phase 3: Completion and Final Validation (March–May 2026)	20
7.2	Quality Assurance Procedures	21
7.2.1	Technical Validation	21
7.2.2	Pedagogical Validation	21
7.2.3	Usability Testing	21
7.3	Documentation and Version Control	22
8	Requirements: Detailed Sub-Requirements	23
8.1	Functional Requirements: Sub-Specifications	23
8.1.1	RF1: Design and Implement EVE-NG Topologies	23
8.1.2	RF2: Implement Vulnerabilities and Attack Scenarios	23
8.1.3	RF3: Create Structured Assessment Materials	24
8.2	Technological Requirements: Sub-Specifications	24
8.2.1	TR1: Virtualisation Infrastructure	24
8.2.2	TR2: Security Tools and Penetration Testing Framework	25
8.2.3	TR3: Scripting and Automation	25
8.3	Non-Functional Requirements: Detailed Subsections	25
8.3.1	NFR1: Performance and Efficiency	25
8.3.2	NFR2: Compliance with Security Frameworks	26

8.3.3	NFR3: Reliability and Availability	27
8.3.4	NFR4: Security and Ethical Compliance	27
8.3.5	NFR5: Usability and Support	27
8.3.6	NFR6: Institutional Sustainability	28
9	Feasibility: Task Breakdown and NIST Risk Assessment	29
9.1	Phase Task Lists	29
9.1.1	Phase 0: Preparation (July–November 2025)	29
9.1.2	Phase 1: Preliminary Project Proposal (October 2025– January 2026)	29
9.1.3	Phase 2: Interim Report (January–March 2026)	30
9.1.4	Phase 3: Final Delivery (March–May 2026)	31
9.2	NIST SP 800-30 Five-Phase Risk Assessment	31
9.2.1	Phase 1: Asset Identification and Scope Definition	32
9.2.2	Phase 2: Threat and Vulnerability Identification	33
9.2.3	Phase 3: Impact and Likelihood Analysis	33
9.2.4	Phase 4: Risk Evaluation and Prioritisation	34
9.2.5	Phase 5: Mitigation Strategies	34
9.3	Detailed Budget Tables	37
9.3.1	Software and Licences	37
9.3.2	Other Operational Costs	37
10	Lab 4: CipherStrike — Instructor Reference	38

1 EVE-NG Installation on Proxmox VE

This appendix documents the configuration used to deploy EVE-NG Community Edition on the Proxmox VE server used for this project. The procedure has been validated and the same steps can be used to reproduce the environment on any compatible Proxmox host. Screenshots and extended notes are available in the companion web documentation at `docs/web/docs/guides/eve_ng_install_proxmox/`.

Virtual Machine Requirements

Parameter	Value used in this project
Hypervisor	Proxmox VE 8.x
CPU type	host (required for nested virtualisation)
vCPU cores	16 (minimum 8)
RAM	32 GB per EVE-NG instance (host upgraded to 64 GB total)
Disk	100 GB VirtIO, SSD-backed Proxmox pool
SCSI controller	VirtIO SCSI
Network	VirtIO, bridged on <code>vmbr0</code> (management); <code>pnet0-pnetN</code> as cloud bridges
BIOS	SeaBIOS (default; OVMF also works)

Table 1.1: EVE-NG VM configuration parameters

Installation Steps

1. Download the EVE-NG Community Edition ISO from the official website.
2. In Proxmox, create a new VM with the parameters in Table 1.1. Ensure the CPU type is set to `host` so that virtualisation extensions (VT-x / AMD-V) are exposed to the guest.
3. Boot from the ISO and follow the Ubuntu Server installation wizard. Assign a static IP address and install the OpenSSH server during setup.

4. After the first reboot, follow the EVE-NG Community post-install wizard (`/opt/unetlab/wrappers/unl_wrapper -a postinstall`) and configure the management interface.
5. Run `apt update && apt upgrade -y` and reboot once more to ensure all EVE-NG patches are applied.
6. Access the web interface at `https://<EVE-NG-IP>/` and change the default credentials immediately.

Proxmox Network Configuration

Two bridge types are used:

- **vmbr0 (bridged)** — management access from the LAN; also used as `pnet0` inside EVE-NG for labs that require connectivity to the homelab network (e.g., Lab 1).
- **vmbr1 (NAT)** — optional internet-facing bridge to keep lab traffic isolated from the LAN while allowing outbound connectivity.

Post-Install Checklist

- ☐ CPU type set to `host` in Proxmox VM options.
- ☐ VM autostart enabled in Proxmox.
- ☐ Snapshot taken after successful EVE-NG installation.
- ☐ Default admin password changed.
- ☐ OS Client Side installed on teacher workstation (HTML5 console option does not require it).
- ☐ Image directories created under `/opt/unetlab/addons/qemu/` and permissions fixed with `unl_wrapper -a fixpermissions`.

2 Lab 1: PEBCAK Corp — Teacher Reference

This appendix provides the deployment reference for Lab 1 (Reconnaissance and Enumeration). It is intended for teachers and system administrators responsible for setting up and resetting the lab environment. The student-facing guide and interactive walkthrough are available at `docs/web/docs/Labs/Lab1/`.

Credentials and Access

Node	Username	Password	Access method
pfSense	admin	pfsense	Web UI / SSH
VyOS	vyos	vyos	EVE-NG console
Server (PEBCAK)	blackmesa	!B!4kM3s\$	SSH (via pfSense DNAT)
PC1	ubuntu	ubuntu	EVE-NG console / VNC
Parrot OS	parrot	parrot	EVE-NG console / VNC

Table 2.1: Lab 1 credentials (teacher reference — do not distribute to students)

Network Segments

Segment	Subnet	Gateway	Purpose
Homelab / WAN	192.168.0.0/24	192.168.0.1	Parrot attacker + pfSense WAN
Net-Link	172.16.0.0/30	pfSense vtnet0	pfSense ↔ VyOS uplink
Users LAN	192.168.10.0/24	192.168.10.x	PC1 internal workstation
Servers LAN	192.168.20.0/24	192.168.20.1	PEBCAK Corp server segment

Table 2.2: Lab 1 network segments

Lab Startup Checklist

1. Start all nodes from the EVE-NG web interface in order: pfSense → VyOS → Server → PC1 → Parrot.

2. Run the bridge script on the EVE-NG host (lost on reboot; persistent via udev rule):

```
1 ip addr add 192.168.20.x/24 dev vnet0_2
2 ip addr add 192.168.10.x/24 dev vnet0_3
3 ip route add 172.16.0.0/30 via 192.168.20.1
```

3. Verify DNS resolution from Parrot: the DNS server on Parrot must point to pfSense's WAN IP so that Lab1 resolves.

```
1 # On Parrot:
2 curl http://lab1          # Should return PEBCAK Corp HTML
```

4. Confirm flag is in place on the Server node:

```
1 sshpass -p '!BL4kM3s$' ssh blackmesa@192.168.20.x cat ~/flag.txt
2 # Expected: FLAG{p3bc4k_s3rv3r_0wn3d}
```

Expected Student Workflow

Step	Action	Tool
1	Navigate to <code>http://lab1</code> in Parrot browser	Firefox
2	Inspect page source; find HTML comment: <code><!-- sometimes simplify and search --></code>	Browser devtools
3	Access <code>http://lab1/pebcak.html</code> ; read SSH credentials	Browser
4	<code>ssh blackmesa@lab1</code> (pfSense DNAT forwards TCP 22)	SSH
5	<code>cat ~/flag.txt</code> — retrieves Flag 1 and pfSense credentials	Terminal
6	SSH to pfSense from Server: <code>ssh admin@172.16.x.x</code>	SSH (optional)

Table 2.3: Lab 1 expected student workflow

Flags

Flag	Value	Location
Flag 1	FLAG{p3bc4k_s3rv3r_0wn3d}	<code>blackmesa/flag.txt</code> on Server
Flag 2	FLAG{pebcak_pfSense_admin}	pfSense admin panel (optional bonus)

Table 2.4: Lab 1 flags

3 Lab 2: SYNAPSE Intelligence Portal — Teacher Reference

This appendix provides the deployment reference for Lab 2 (Web Application Vulnerabilities). It is intended for teachers and administrators. The application source code, Docker Compose files, and interactive walkthrough are at `docs/web/docs/Labs/Lab2/`.

Credentials and Access

System	Username	Password	Role
pfSense-LAB2	admin	pfsense	Firewall management
VyOS-LAB2	vyos	vyos	Router console
Server-A OS	ubuntu	S3rv3rA	SSH access
Server-A Portal	admin	Admin@Synapse2024	Web application admin
Server-A Portal	guest	guest123	Web application guest
Server-B OS	ubuntu	S3rv3rB	SSH access
Server-B DataVault	operator	D4t4V4ult#2024	SSH access
Parrot-LAB2	lab2	L4b2	Attacker node

Table 3.1: Lab 2 credentials (teacher reference — do not distribute to students)

Vulnerability Chain and Flag Locations

Step	OWASP	Vuln	Implementation	Flag
1	A03:2021	Stored XSS	safe filter in Jinja2 template; /comments endpoint	—
2	A07:2021	Cookie theft	HttpOnly=False; victim bot (Playwright) auto-visits comments	—
3	A07:2021	Broken auth	Cookie format ID:ROLE:USERNAME; unsigned; forgeable	—
4	A03:2021	SQL injection	/search?q= uses string concatenation	FLAG 1
5	A01:2021	Admin access	/admin panel reveals Server-B credentials	FLAG 2
6	A08:2021	YAML deser.	Server-B /preview uses yaml.UnsafeLoader	—
7	A08:2021	RCE	!!python/object/apply:os.system payload via POST	FLAG 3

Table 3.2: Lab 2 vulnerability chain and flag locations

Lab Startup Procedure

1. After EVE-NG host reboot, restore bridge addresses:

```
1 bash /usr/local/bin/lab2-bridges-up.sh
```

2. Start the SYNAPSE Portal on Server-A:

```
1 sshpass -p S3rv3rA ssh ubuntu@192.168.30.x
2 cd /opt/synapse && docker-compose up -d
3 docker-compose ps # expect: flask, nginx, victim — all Up
```

3. Start the DataVault on Server-B:

```
1 sshpass -p S3rv3rB ssh ubuntu@192.168.30.x # Server-B IP
2 cd /opt/datavault && sudo docker compose up -d
3 sudo docker compose ps # expect: datavault-flask Up on :80
```

4. Configure Parrot static IP and verify connectivity:

```
1 # On Parrot (10.0.40.x/24, GW 10.0.40.x):
2 nmcli con mod "Wired connection 1" ipv4.method manual \
3   ipv4.addresses 10.0.40.x/24 ipv4.gateway 10.0.40.x
4 nmcli con up "Wired connection 1"
```

```
5 | curl http://192.168.30.x    # Should return SYNAPSE Portal HTML
```

Note: pfSense-LAB2 is installed in UEFI mode. If it fails to boot, use the start script: `/usr/local/bin/pfsense-lab2-start.sh`. Server-A uses `docker - compose` (v1, hyphen); Server-B uses `docker compose` (v2, space).

4 Lab 3: HELIX Systems — Teacher Reference

This appendix provides the deployment and pre-staging reference for Lab 3 (Incident Response and Log Forensics). It documents all accounts, the pre-staged attack timeline, and the evidence artefacts that students are expected to find. The step-by-step investigation walkthrough is at docs/web/docs/Labs/Lab3/walkthrough/.

Accounts

System	Username	Password	Role
pfSense-LAB3	admin	pfSense	Firewall management
VyOS-LAB3	vyos	vyos	Router console
Server-Web	analyst	An@lys@2024	Student entry account
Server-Web	devops	D3v0ps#2023	Compromised account (brute-forced)
Server-Web	maint	—	Backdoor account (attacker-created)
Server-DB	analyst	An@lys@2024	Internet access
Server-DB	devops	D3v0ps#2023	Compromised (password reuse)

Table 4.1: Lab 3 accounts (teacher reference)

Pre-Staged Attack Timeline

Time	Host	Event
03:05–03:16	Server-Web	25 failed SSH attempts for devops from 185.220.101.47
03:17	Server-Web	Successful SSH login as devops
03:18	Server-Web	sudo /usr/bin/find — privilege escalation to root via GTFOBins
03:19	Server-Web	useradd maint + /etc/sudoers.d/maint — backdoor + persistence
03:21	Server-DB	SSH login as devops from 192.168.50.10 (lateral movement)
03:22	Server-DB	Access to clients.db; .exfil_marker created

Table 4.2: Lab 3 pre-staged attack timeline

Evidence Artefact Locations

Artefact	Path	Host
SSH brute-force log	/var/log/auth.log	Server-Web
Privilege escalation	/var/log/auth.log (sudo entry)	Server-Web
Backdoor account	/etc/passwd, /etc/sudoers.d/maint	Server-Web
Sudoers misconfiguration	/etc/sudoers.d/devops	Server-Web
Bash history	/home/devops/.bash_history	Server-Web
Lateral movement log	/var/log/auth.log	Server-DB
Exfiltration marker	/opt/helix/data/.exfiltration	Server-DB
Sensitive data	/opt/helix/data/client_secrets	Server-DB

Table 4.3: Lab 3 evidence artefact locations

Student Entry Point

Students connect via SSH to pfSense's WAN IP; pfSense DNAT forwards TCP 22 directly to Server-Web:

```

1 ssh analyst@pfSense-WAN-IP>
2 # Password: Analyst2024
3 # Lands on: analyst@helix-web:~$

```

No offensive tools are required. Students use only standard Unix utilities: `grep`, `awk`, `less`, `cat`, `find`, `journalctl`.

Lab Reset Procedure

To restore the lab to its initial forensic state after a student session:

```

1 # Restore EVE-NG snapshot to pre-exercise state
2 # (from EVE-NG host or web interface)
3 unl_wrapper -a ... # see web docs for exact snapshot name

```

Alternatively, revert both Server-Web and Server-DB QCOW2 images from the pre-staged backup stored at `/opt/unetlab/addons/qemu/lab3-backups/`.

5 Cybersecurity Training Platform Descriptions

This appendix provides detailed descriptions of the seven cybersecurity training platforms reviewed in Chapter 2 (State of the Art). Each subsection discusses the platform's approach, strengths, and limitations relevant to curriculum-aligned institutional deployment. A comparative summary is provided in Table ??.

5.1 HackTheBox

HackTheBox [?] is one of the most widely recognised online platforms for penetration testing practice, offering a large library of intentionally vulnerable machines and Capture-the-Flag challenges categorised by difficulty. The platform benefits from a strong community and industry recognition, making it a common supplementary resource for professional certifications. However, its challenges are isolated: there is no network topology connecting systems, no formal curriculum mapping to specific degree programmes, and the platform cannot be deployed on institutional infrastructure, which limits customisation and assessment integration.

5.2 TryHackMe

TryHackMe [?] adopts a gamified, beginner-friendly approach, combining structured learning paths with guided interactive labs accessible entirely from a browser. Its low barrier to entry and progressive structure make it effective for initial skill development. However, the platform is externally managed and subscription-dependent, scenarios are simplified to ensure completion within short timeframes, and it does not support local institutional deployment or integration with academic learning management systems.

5.3 Cisco Networking Academy

Cisco Networking Academy [?] provides institution-backed curriculum aligned with professional certifications (CCNA, CyberOps) and uses Packet Tracer for network simulation. Its formal structure and vendor support are strengths for foundational networking education. However, the programme is primarily defensive and networking-oriented, Packet Tracer does not support full operating system simulation or security tool integration, and the curriculum is tightly coupled to Cisco technologies.

5.4 SEED Labs

SEED Labs [?] is an open-source, research-backed collection of Docker-based security exercises covering specific concepts such as buffer overflows, SQL injection, and cryptographic protocols. The academic rigour and local deployability are significant advantages. Nevertheless, each lab is conceptually isolated, there is no structured penetration testing workflow spanning multiple systems, and significant teacher customisation is required to integrate SEED exercises into a complete curriculum.

5.5 Game of Active Directory (GOAD)

GOAD [?] provides an Infrastructure-as-Code vulnerable Active Directory environment designed for realistic Windows domain penetration testing. Its use of Terraform and Vagrant enables reproducible deployment and the scenarios reflect real corporate environments. The platform is, however, narrowly focused on Windows AD: it does not cover reconnaissance, web application vulnerabilities, or network traffic analysis, and it lacks formal teacher resources or curriculum structure.

5.6 VulnHub

VulnHub [?] is a community-driven repository of downloadable vulnerable virtual machines for local deployment. Its open, self-directed nature

suits independent learners well, but the absence of structured progression, inconsistent quality across community-contributed machines, and lack of automation for deployment and reset make institutional integration impractical without significant teacher effort.

5.7 Hack4u Academy

Hack4u Academy [?] stands out for its emphasis on penetration testing methodology over tool memorisation, offering structured video-based courses with integrated practical exercises. Its curriculum is well-designed and the community active. The platform is, however, subscription-based and externally hosted, making local institutional deployment and automated large-scale assessment impossible.

6 Objectives: Deliverables, Detailed KPIs, and Audience

This appendix complements Chapter ?? with the full deliverable specifications per specific objective, the detailed per-category KPI tables, and the extended audience analysis that frames the project's reuse potential.

6.1 Deliverables by Specific Objective

6.1.1 OE1: Design and Implement EVE-NG Labs

- 4 .unl topology files (Lab 1: implemented; Labs 2–4: planned)
- 8–12 optimised virtual machine images
- Technical configuration documentation for each lab

6.1.2 OE2: Develop Automation Scripts

- Bash/Python utility scripts for common tasks (e.g. bridge configuration, ISO upload, node connectivity checks)
- Scripts documented with usage instructions in the repository

6.1.3 OE3: Develop Structured Teaching Material

- Student lab guide PDF per lab — task description, objectives, hints, and tools reference
- Teacher resolution guide PDF per lab — full walkthrough, flag values, rubric (100 pts), and common errors
- Technical configuration reference per lab in the repository

6.1.4 OE4: Validate Usability and Educational Effectiveness

- Pilot session report with timing and usability observations
- Post-session survey results (satisfaction, difficulty, clarity)
- List of improvements implemented based on pilot feedback

Pilot note: The pilot group comprises 4–5 individuals from the GEISI student population, potentially including participants with documented learning difficulties (e.g. dyslexia), to assess the effectiveness of the accessibility measures incorporated in the lab documentation.

6.2 Detailed KPI Tables

The following tables break down the consolidated KPIs from Table ?? by category for reference.

6.2.1 Technical Indicators

Metric	Objective	Measurement Method
Lab deployment time	≤ 2 min	Automated timing
VM boot success rate	$\geq 95\%$	System logs + validation scripts
Full reset time	≤ 30 s	Scripts with timestamps
Scenario reproducibility	100%	Automated tests

Table 6.1: Technical KPIs for EVE-NG lab deployment

6.2.2 Educational Indicators

Metric	Objective	Measurement Method
Lab resolution time	90–120 min	Time tracking during pilot
Lab completion (pilot)	All participants	Observation during pilot sessions
User satisfaction	$\geq 4/5$	Post-use survey
Key concept understanding	$\geq 80\%$ correct	Assessment questionnaire

Table 6.2: Educational KPIs for lab effectiveness

6.2.3 Quality Indicators

Metric	Objective	Measurement Method
Documentation coverage	100% of features	Verification checklist
Critical errors	0	Exhaustive testing
EVE-NG version compatibility	100%	Tests in multiple environments
Interface usability	$\geq 4/5$	UX heuristic evaluation

Table 6.3: Quality KPIs for lab package

6.3 Extended Audience Analysis

6.3.1 Secondary End Users

- **Cybersecurity Faculty:** Other teachers interested in reusing the material for related courses
- **Postgraduate Students:** Possible extensions of the material for advanced levels

6.3.2 Potential Audience Beyond Tecnocampus

Internal Scope (Tecnocampus)

- Other courses related to cybersecurity
- Research projects in computer security
- Continuing education and professional certifications

External Scope

- Educational institutions with training programmes in cybersecurity
- Specialised vocational training centres
- Companies with internal security training programmes

7 Methodology: Detailed Phase Activities and Quality Assurance

This appendix expands the phase summaries in Chapter ?? with full activity lists, quality assurance procedures, and documentation and version control practices.

7.1 Detailed Phase Activity Lists

7.1.1 Phase 0: Preparation and Infrastructure (July–November 2025)

- HomeLab infrastructure initialisation (Proxmox[?], networking, storage)
- Development environment setup (VSCode[?], LaTeX[?], Git)
- Security tools installation (Parrot OS Security[?], Metasploit[?], Burp Suite[?])
- Technical documentation and baseline establishment

Deliverables: Fully operational virtualisation infrastructure; development environment configuration; baseline documentation of system architecture.

Duration: 105 hours **Status:** Completed (100%)

7.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026)

1. **SMART Objectives Definition (4h)** — Specification of Main Objective (MO): develop a reusable teaching package. Definition of 4 Specific Objectives (OE1–OE4) with measurable KPIs. Alignment with institutional GEIS curriculum requirements.

2. **State of the Art Analysis (15h)** — Comprehensive review of 7 major cybersecurity training platforms (HackTheBox, TryHackMe, Cisco Academy, SEED Labs, GOAD, VulnHub, Hack4u). Identification of pedagogical gaps. Comparative analysis matrix across multiple dimensions.
3. **Functional and Non-Functional Requirements (10h)** — RF1: design and implement 4 EVE-NG topologies (.unl files). RF2: implement vulnerabilities per PTES[?], OWASP[?], NIST[?], and CEH[?] standards. RF3: create structured assessment materials and rubrics. NFR1–NFR3 covering performance, framework compliance, and institutional sustainability.
4. **Pentesting Methodology and Validation Framework (8h)** — Adaptation of PTES for educational context. Definition of ethical hacking principles (EC-Council CEH). Mapping of attack phases to lab objectives. Validation criteria for security effectiveness.
5. **Design Decisions Justification (5h)** — Rationale for EVE-NG selection vs alternative platforms. Technology stack justification (Parrot OS Security, Metasploit, Burp, OWASP ZAP). Pedagogical approach: hands-on labs with automation.
6. **Risk Analysis and Mitigation Planning (15h)** — Identification of 12+ project risks (technical, pedagogical, institutional). Assessment of impact and probability. Definition of mitigation strategies.
7. **Resource Inventory and Allocation (5h)** — Hardware requirements: dual-socket server with Proxmox. Software stack: EVE-NG Community Edition, Parrot OS Security images, security tools. Human resources: student (~820h total), faculty supervisor. Budget considerations and open-source alternatives.
8. **Gantt Chart and Critical Path Analysis (20h)** — Development of detailed project timeline with 37 tasks. Identification of 13 critical path tasks. CSV export and interactive web-based visualisation. Progress tracking and milestone definition.

Deliverables: Formal project proposal (Preliminary Project Proposal); detailed Gantt chart with critical paths; risk register and mitigation plan; requirements specification (functional and non-functional).

Duration: 82 hours **Status:** Completed **Milestone Deadline:** January 16, 2026

7.1.3 Phase 2: Implementation and Interim Validation (January–March 2026)

1. **Lab 1: Reconnaissance Development** (14h research + 20h dev)
— Scenario: intelligence gathering phase using PTES framework. Tools: Nmap[?], passive reconnaissance techniques. Topology: multi-tier network with passive monitoring capabilities. Validation: successful execution of reconnaissance workflow.
2. **Lab 2: Web Vulnerabilities Development** (14h research + 25h dev)
— Scenario: OWASP Top 10 vulnerability exploitation. Tools: Burp Suite, OWASP ZAP, manual testing techniques. Topology: web application server with vulnerable services. Validation: exploitation success and reporting accuracy.
3. **Lab 3: Incident Response Development** (14h research + 25h dev)
— Scenario: log forensics and incident response (NIST SP 800-61[?] standards). Tools: Wireshark[?], tcpdump, network enumeration tools. Topology: multi-segment network with pre-staged attack artefacts. Validation: successful timeline reconstruction and flag recovery.
4. **Pilot Validation with Users** (20h + 6h revision) — Informal testing sessions with 4–5 GEISI students conducted as each lab is completed. Usability observations and timing notes. Post-session survey on satisfaction, difficulty, and guide clarity. Adjustments applied based on participant feedback.
5. **Topology Adjustments and Optimisation** (30h) — Performance tuning based on pilot feedback. VM image optimisation and deployment time reduction. Network configuration refinement for stability.
6. **Teaching Material Production** (40h) — Student lab guide PDF per lab: task description, objectives, hints, and tool reference. Teacher resolution guide PDF per lab: one possible walkthrough, flag values,

assessment rubric, and common errors. Technical configuration reference per lab in the repository.

Deliverables: 3 fully functional EVE-NG labs; student and teacher guides; pilot validation report; technical configuration references.

Duration: 188 hours **Status:** Completed **Milestone Deadline:** March 20, 2026

7.1.4 Phase 3: Completion and Final Validation (March–May 2026)

1. **Lab 4: Privilege Escalation Development** (16h research + 30h dev) — Scenario: post-exploitation and privilege escalation (CEH framework). Tools: Metasploit, manual exploitation, system enumeration. Topology: hardened systems with privilege escalation vectors. Validation: successful privilege elevation and system compromise.
2. **Comprehensive Penetration Testing** (30h Round 1 + 25h Round 2) — Round 1: sequential testing of Labs 1, 2, and 3. Round 2: integrated testing across all 4 laboratories. Documentation of all findings and remediation strategies.
3. **Automation Scripts Development** (30h) — Utility scripts (Bash/Python) for recurring lab management tasks: bridge configuration, ISO upload, and connectivity check scripts. Scripts documented and tested in the project EVE-NG environment.
4. **Final User Validation and Iteration** (18h) — Final informal validation sessions with the pilot group (4–5 participants). Collection of post-session survey data and usability notes. Implementation of final adjustments based on feedback.
5. **Bachelor's Thesis Report Writing and Finalisation** (90h) — Phase I (15h): Introduction, Objectives, Literature Review. Phase II (40h): Methodology, Requirements, Implementation Results. Phase III (35h): Validation, Analysis, Conclusions. Final revision, bibliography integration, and publication preparation (35h).

Deliverables: 4 fully functional labs; utility automation scripts; final bachelor's thesis report; pilot validation report; deployment package ready for institutional use.

Duration: 315 hours **Status:** In progress **Final Milestone Deadline:** May 27, 2026

7.2 Quality Assurance Procedures

7.2.1 Technical Validation

- Automated testing of lab deployment and reset
- Manual security testing and penetration verification
- Performance benchmarking against KPI targets
- Compatibility testing across EVE-NG versions

7.2.2 Pedagogical Validation

- Alignment with GEISI learning outcomes
- Assessment of student understanding through rubrics
- Effectiveness in achieving educational objectives
- Feedback from faculty and student participants

7.2.3 Usability Testing

- User interface and documentation clarity
- Accessibility and ease of deployment
- Support and documentation adequacy
- User satisfaction scoring ($\geq$ 4/5 target)

7.3 Documentation and Version Control

- GitHub repository (TheEgea/TFG) for all code, scripts, and documentation, licensed under CC BY-NC-SA 4.0
- LaTeX (XeLaTeX + OpenDyslexic) for formal academic documentation (Volume I and Volume II)
- MkDocs Material for the operational web documentation site
- Versioning system for all deliverables with semantic commit messages
- Change logs and improvement tracking through pull requests
- Deployment guides and runbooks per lab

8 Requirements: Detailed Sub-Requirements

This appendix expands the requirements in Chapter ?? with the full sub-requirement specifications for each functional requirement (RF), technological requirement (TR), and non-functional requirement (NFR).

8.1 Functional Requirements: Sub-Specifications

8.1.1 RF1: Design and Implement EVE-NG Topologies

- **RF1.2:** Each topology must include:
 - Minimum 5–8 virtual machines per lab
 - Realistic network architecture (DMZ, internal networks, monitoring zones)
 - Vulnerable services configured per OWASP and NIST standards
 - Network segmentation with VLANs where appropriate
- **RF1.3:** EVE-NG compatibility requirements:
 - .unl file format compatible with EVE-NG Community Edition
 - Additional compatibility with EVE-NG Professional Edition
 - KVM hypervisor support (Ubuntu 20.04+ LTS)
 - Minimum host specification: 16 GB RAM, 200 GB storage
- **RF1.4:** Topology documentation:
 - Network diagram for each lab (logical and physical views)
 - IP addressing scheme documented
 - Service inventory and port mappings
 - Attack surface identification and justification

8.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

- **RF2.5:** Vulnerability implementation standards:

- All vulnerabilities must be fixable by administrators
- No permanently broken or unrecoverable systems
- Clear reset procedure for each vulnerability state
- Automated scanning tools must successfully identify vulnerabilities

8.1.3 RF3: Create Structured Assessment Materials

- **RF3.3: Support materials:**
 - Student lab manuals with step-by-step guidance
 - Teacher guides with deployment and customisation instructions
 - Expected outcomes and common pitfalls documentation
 - Troubleshooting guides for common technical issues
- **RF3.4: Validation materials:**
 - Automated validation scripts to verify successful lab completion
 - Manual verification procedures for educators
 - Documentation of success indicators

8.2 Technological Requirements: Sub-Specifications

8.2.1 TR1: Virtualisation Infrastructure

- **TR1.2: Hypervisor:**
 - Primary: KVM (Kernel-based Virtual Machine)
 - Host OS: Ubuntu Server 20.04 LTS or 22.04 LTS
 - QEMU integration for VM management
 - Nested virtualisation support for advanced scenarios
- **TR1.4: Network configuration:**

- Layer 2 switching capabilities within EVE-NG
- VLAN support for network segmentation scenarios
- IPv4 and IPv6 support

8.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.4: Compliance and documentation:**
 - All tools must be freely available or have educational licences
 - Open-source preference for long-term sustainability
 - Tool version specifications documented
 - Legal and ethical use guidelines included

8.2.3 TR3: Scripting and Automation

- **TR3.3: Integration points:**
 - EVE-NG API integration for programmatic lab management
 - Webhook support for event-driven automation
 - Container orchestration (Docker [?]) for microservices
 - Version control integration (Git, GitHub)

8.3 Non-Functional Requirements: Detailed Subsections

8.3.1 NFR1: Performance and Efficiency

NFR1.1 – Deployment Performance:

- Lab deployment time ≤ 2 minutes from topology load to ready state
- VM boot time ≤ 90 seconds per VM (95% of cases)
- Reset operation ≤ 30 seconds to restore to initial state

- Concurrent lab instances: support minimum 5 simultaneous labs

NFR1.2 – Resource Optimisation:

- VM image sizes ≤ 5 GB per image (compressed)
- Memory footprint ≤ 6 GB per lab instance during execution
- Minimal external dependencies (all local simulation)

NFR1.3 – Scalability:

- Current infrastructure supports 3–4 concurrent instances on the project HomeLab server
- Architecture horizontally scalable: additional EVE-NG nodes can be added
- EVE-NG Professional Edition supports larger concurrent deployments

8.3.2 NFR2: Compliance with Security Frameworks

NFR2.1 – NIST Cybersecurity Framework [?]: Core Functions (Identify, Protect, Detect, Respond, Recover) mapped to lab exercises; assessment procedures aligned with NIST guidelines.

NFR2.2 – CIS Critical Controls [?]: Controls 1–18 incorporated where pedagogically appropriate; assessment rubrics aligned with CIS benchmarks; remediation procedures follow CIS recommendations.

NFR2.3 – OWASP Standards [?, ?]: OWASP Top 10 fully covered; OWASP Testing Guide methodology integrated; development practices reflected in lab design.

NFR2.4 – Penetration Testing Standards [?, ?]: PTES phases mapped to labs; CEH curriculum alignment; ethical hacking principles strictly enforced.

8.3.3 NFR3: Reliability and Availability

NFR3.1 – System Reliability:

- MTBF $\geq$ 100 hours continuous operation
- MTTR $\leq$ 5 minutes
- Scenario reproducibility: 100% (deterministic behaviour)
- Zero critical bugs at release

NFR3.2 – Data Integrity: No unintended data loss during lab operations; snapshot and restore functionality; backup procedures for sensitive configurations.

NFR3.3 – Availability: Planned maintenance windows weekends only; system availability $\geq$ 99% during instructional periods; graceful degradation if resources become limited.

8.3.4 NFR4: Security and Ethical Compliance

NFR4.1 – Educational Sandbox Isolation: Complete network isolation from production systems; no outbound internet access from lab environments; contained attack scenarios (no external targets).

NFR4.2 – Access Control and Accountability: Role-based access control (RBAC) for lab management; audit logs of all student actions where applicable; clear identification of authorised vs. unauthorised activities.

NFR4.3 – Ethical Guardrails: Mandatory ethics training before lab access; written agreements regarding responsible use; monitoring and escalation procedures for suspicious activities.

8.3.5 NFR5: Usability and Support

NFR5.1 – Ease of Use: Average faculty deploy time 5–10 minutes; intuitive web interface with clear navigation; minimal technical prerequisites for students.

NFR5.2 – Documentation and Support: Comprehensive user manuals and quick-start guides; FAQ documentation; responsive support channel.

NFR5.3 – Accessibility: WCAG 2.1 Level AA compliance for web interfaces; screen reader compatibility; keyboard navigation support; clear contrast ratios.

8.3.6 NFR6: Institutional Sustainability

NFR6.1 – Long-term Maintainability: All components open-source or freely available; no vendor lock-in; source code in GitHub with clear documentation; backward compatibility with older EVE-NG versions where feasible.

NFR6.2 – Curriculum Alignment: Explicit mapping to GEISI degree learning outcomes; integration with existing course structures; flexible customisation for different instructional contexts.

NFR6.3 – Scalability for Institutional Adoption: Deployable on institutional infrastructure (not SaaS-dependent); support for multiple faculty deployments simultaneously; resource reservation and scheduling capabilities.

9 Feasibility: Task Breakdown and NIST Risk Assessment

This appendix complements Chapter ?? with the full per-phase task lists (task IDs, estimated hours, critical path flags) and the five-phase NIST SP 800-30 risk assessment that underpins the risk matrix in that chapter.

9.1 Phase Task Lists

9.1.1 Phase 0: Preparation (July–November 2025)

- Task 0.1: HomeLab Initialisation Setup (64h)
- Task 0.2: VSCode Setup for thesis development (18h)
- Task 0.3: Parrot OS Security Setup and Basic Tools (15h)
- Task 0.4: Home Lab As-Built Documentation (8h)

Duration: 105 hours **Status:** Completed (100%)

9.1.2 Phase 1: Preliminary Project Proposal (October 2025–January 2026)

- Task 1.0: Thesis Development (22h)
- Task 1.1: SMART Objectives Definition (4h)
- Task 1.2: State of the Art: Frameworks and Tools (15h)
- Task 1.3: Functional and Non-Functional Requirements (10h)
- Task 1.4: Pentesting Methodology and Validation (8h) — **CRITICAL**
- Task 1.5: Design Decisions and Justification (5h)
- Task 1.6: Gantt Chart Development (20h)

- Task 1.7: Risk Analysis and Mitigation Plan (15h)
- Task 1.8: Resource Inventory (5h)
- **Milestone 1.10: Preliminary Project (40h) — Delivery: January 16, 2026**

Duration: 82 hours **Status:** Completed

9.1.3 Phase 2: Interim Report (January–March 2026)

- Task 2.1: Lab 1 Research — Reconnaissance (14h) — **CRITICAL**
- Task 2.2: Lab 2 Research — Web Vulnerabilities (14h) — **CRITICAL**
- Task 2.3: Lab 3 Research — Network Analysis/IR (14h) — **CRITICAL**
- Task 2.4: Technical Write-ups Labs 1 & 2 (13h)
- Task 2.5: Topology Adjustments (30h)
- Task 2.6: Pre-validation with Pilot Users (20h) — **CRITICAL**
- Task 2.6.1: Pre-validation Revision (6h) — **CRITICAL**
- Task 2.7: Analyse and Write Hackathon Results (17h)
- Task 2.8: Interim Report Formalisation (40h)
- Task 2.9: Set Up Servers, VMs, and EVE-NG (20h)
- Task 2.10: Backup and Documentation of EVE-NG (10h)
- **Milestone 2.11: Intermediate Thesis (50h) — Delivery: March 20, 2026**

Duration: 212 hours **Status:** Completed

9.1.4 Phase 3: Final Delivery (March–May 2026)

- Task 3.1: Lab 4 Research — Post-Exploitation (16h) — **CRITICAL**
- Task 3.2: Lab 3 Validation with Users (12h)
- Task 3.3: Technical Write-ups Labs 3 & 4 (19h)
- Task 3.4: Penetration Testing Round 1 (Labs 1–2–3) (30h) — **CRITICAL**
- Task 3.5: Penetration Testing Round 2 (All Labs) (25h) — **CRITICAL**
- Task 3.6: Python Automation Scripts (30h)
- Task 3.7: Final Validation with Pilot Users (18h)
- Task 3.8: Final Review and Project Polishing (25h)
- Task 3.9: Bachelor's Thesis Report Writing I (15h) — **CRITICAL**
- Task 3.10: Bachelor's Thesis Report Writing II (40h) — **CRITICAL**
- Task 3.11: Bachelor's Thesis Report Writing III (35h) — **CRITICAL**
- Task 3.12: Final Revision (15h)
- Task 3.13: Final Report Preparation (20h)
- **Milestone 3.14: Final Report (5h) — Delivery: May 27, 2026**

Duration: 315 hours **Status:** In progress

9.2 NIST SP 800-30 Five-Phase Risk Assessment

This section details the systematic risk assessment following NIST SP 800-30[?]. The summary risk matrix is provided in Table ?? in Chapter ??.

9.2.1 Phase 1: Asset Identification and Scope Definition

Critical assets are categorised by CIA triad:

Hardware Assets:

- HomeLab server (2× Xeon E5-2680v4, 64 GB RAM) — Availability critical
- Development workstations (laptop + desktop) — Integrity critical
- Network equipment (router, switch) — Availability critical
- External storage (backup) — Confidentiality + Availability critical

Software Assets:

- EVE-NG Community Edition — Availability critical
- Parrot OS Security distribution — Integrity critical
- Vulnerable VMs (Metasploitable, DVWA) — Integrity critical
- Development tools (VSCode, Git, LaTeX) — Integrity critical

Data Assets:

- Thesis source code and documentation — Confidentiality + Integrity critical
- Lab topology configurations — Integrity + Availability critical
- Student learning materials — Availability critical
- Project research findings — Confidentiality + Integrity critical

Scope: Infrastructure and technical risks within the controlled laboratory environment. **Excluded:** organisational, regulatory, and external threats beyond the academic context.

9.2.2 Phase 2: Threat and Vulnerability Identification

Threat categories (NIST SP 800-30):

- **Hardware Threats:** Disk failures, power supply degradation, CPU overheating, memory exhaustion
- **Software/Configuration Threats:** Outdated VM images, incompatible versions, configuration drift, EVE-NG crashes
- **Knowledge Gap Threats:** Insufficient expertise in advanced networking, virtualisation optimisation, security best practices
- **Resource Constraint Threats:** Limited RAM for simultaneous VM operations, storage capacity limits
- **Integration Threats:** EVE-NG network compatibility issues, tool interoperability
- **Data Loss Threats:** Accidental deletion, corruption, hardware failure

Key vulnerabilities:

- Aging server components (end-of-life approaching)
- Single point of failure — no infrastructure redundancy
- EVE-NG Community Edition has no commercial support
- Limited RAM capacity for production-scale virtualisation
- Inadequate documentation of network topology state
- Limited backup redundancy (single backup location)

9.2.3 Phase 3: Impact and Likelihood Analysis

Scales follow NIST SP 800-30: Probability (Low <25%, Medium 25–75%, High >75%); Impact (Low: minor delays, Medium: project delays, High: project jeopardy).

Risk Scenario			Prob.	Impact	Level	Primary Asset
EVE-NG Performance Issues			Medium	High	HIGH	Infrastructure, Labs
VM Image Incompatibility			Medium	Medium	MEDIUM	Development, Labs
Knowledge Gaps (advanced net.)			Medium	Medium	MEDIUM	Development, Timeline
Hardware Degradation / Disk Failure			Low	High	MEDIUM	Infrastructure
Data Loss	(insufficient backup)		Low	High	MEDIUM	Documentation
Configuration Gaps	Documenta-		Medium	Medium	MEDIUM	Operations

Table 9.1: Risk assessment matrix with probability and impact (NIST SP 800-30)

9.2.4 Phase 4: Risk Evaluation and Prioritisation

HIGH Priority: EVE-NG Performance Issues — infrastructure bottlenecks causing lab functionality failures. Medium probability, high impact. Requires immediate resource optimisation and monitoring implementation.

MEDIUM Priority:

1. VM Incompatibility — disparate image formats causing integration problems
2. Knowledge Gaps — experience gaps in advanced networking/virtualisation
3. Configuration Documentation — inadequate topology documentation
4. Data Loss — insufficient backup redundancy (single point of failure)

LOW Priority: Hardware Degradation — aging components manageable through preventive maintenance.

9.2.5 Phase 5: Mitigation Strategies

EVE-NG Performance Issues (REDUCTION):

- **Hardware dimensioning:** 64 GB RAM dedicated to the server running EVE-NG (32 GB proved insufficient — additional module acquired, ~80 €)
- **Resource optimisation:** memory tuning, VM suspension when idle
- **Documented fallback:** VirtualBox as alternative hypervisor if critical failure
- **Result:** Probability reduced from Medium to Low

VM Incompatibility (PREVENTION):

- **Early testing:** validate all images in target environment before use
- **Format standardisation:** QCOW2 for EVE-NG compatibility
- **Alternative repositories maintained** (VulnHub, GitHub)
- **Result:** Probability reduced from Medium to Low

Knowledge Gaps (PREVENTION + REDUCTION):

- **Structured learning:** Task 1.2 (15h) comprehensive framework research
- **Continuous upskilling** during lab research phases (Tasks 2.1–3.1)
- **Expert consultation** with tutor Pere Vidiella i Catalan
- **Technical references:** NIST[?], OWASP[?]
- **Result:** Impact reduced from Medium to Low-Medium

Configuration Documentation (PREVENTION):

- **Continuous documentation:** update topology diagrams during development
- **Automated export:** EVE-NG built-in topology export
- **Version control:** all network configs via Git repository

- **Result:** Probability reduced from Medium to Low

Data Loss (REDUCTION):

- Daily automated encrypted backups to external storage
- Version control: full Git history on GitHub
- Off-site cloud backup (student account)
- **Result:** Probability Low, Impact Low

Hardware Degradation (PREVENTION):

- Quarterly hardware diagnostics (disk health, temperature monitoring)
- Automated S.M.A.R.T. alerts for disk errors and CPU temperature thresholds
- Component procurement plan with 6-month horizon for critical parts

Risk Assessment Conclusion: All identified risks have been systematically evaluated and assigned concrete mitigation strategies. No individual risk is considered project-blocking.

9.3 Detailed Budget Tables

9.3.1 Software and Licences

Resource	Licence Type	Unit	Total
EVE-NG Community Edition	Free/Open Source	0 €	0 €
Parrot OS Security	Free/Open Source	0 €	0 €
Proxmox VE	Free/Open Source	0 €	0 €
Metasploitable VMs	Free/Open Source	0 €	0 €
DVWA	Free/Open Source	0 €	0 €
Visual Studio Code	Free/Open Source	0 €	0 €
LaTeX (TeX Live)	Free/Open Source	0 €	0 €
Git + GitHub (Student)	Free/Student	0 €	0 €
Burp Suite Community	Free	0 €	0 €
Wireshark	Free/Open Source	0 €	0 €
SUBTOTAL Software			0 €

Table 9.2: Software and licence costs (all free/open-source for educational use)

9.3.2 Other Operational Costs

Concept	Cost
Electricity ($820 \text{ h} \times 0.215 \text{ kWh} \times 0.20 \text{ €/kWh}$)	35 €
Internet connectivity (9 months $\times 40 \text{ €/month}$)	360 €
Contingency fund (5% of total)	747 €
TOTAL Other Costs	1,142 €

Table 9.3: Detailed breakdown of additional operational costs

Note: The total budget summary in Chapter ?? (Table ??) uses a rounded figure of 1,142 € for other costs, reflecting the exclusion of the contingency fund from the base estimate.

10 Lab 4: CipherStrike — Instructor Reference

This appendix provides the deployment and configuration reference for Lab 4 (Cryptography and Steganography CTF). It is intended for instructors and system administrators responsible for setting up, resetting, and grading the lab environment. The student-facing challenge guide is available at `docs/web/docs/Labs/Lab4/`.

Credentials and Access

Node	Username	Password	Access method
pfSense-LAB4	admin	pfSense	Web GUI / SSH
VyOS-LAB4	vynos	vynos	EVE-NG console
ServerA	admin	N3x5t3r	SSH 10.10.1.10
ServerB	sysop	S3c5r3t	SSH 10.10.2.10
ServerC	devops	fa681b59855d	SSH 10.8.0.5d10 (derived — see below)
Defender	—	—	Monitoring only

Table 10.1: Lab 4 credentials (instructor reference — do not distribute to students)

ServerC password derivation: The ServerC password is not given directly to students. It is the first 12 hex characters of the MD5 hash of the concatenation of flags A5 and B5: `md5(A5_flag + B5_flag)[:12] = fa681b59855d`. Students must solve modules A and B completely before they can access ServerC. The decoy password `D3vOps24!` appears as a false hint in some challenge materials to test attention to detail.

Network Segments

Segment	Subnet	Gateway	Purpose
WAN (Homelab)	192.168.0.0/24	192.168.0.1	pfSense WAN (DHCP, 192.168.0.18)
Net-Link	192.168.1.0/24	192.168.1.1	pfSense LAN ↔ VyOS uplink
Zone-A	10.10.1.0/24	10.10.1.1	ServerA (Cryptography)
Zone-B	10.10.2.0/24	10.10.2.1	ServerB (Steganography)
Zone-C	10.10.3.0/24	10.10.3.1	ServerC (Advanced Mix)
Zone-Defense	10.10.4.0/24	10.10.4.1	Defender (monitoring)

Table 10.2: Lab 4 network segments

Challenge Inventory

ServerA — Cryptography Module

All challenge files reside in `/home/admin/mensajes/` on ServerA.

ID	File(s)	Technique	Flag
A1	A1_mensaje_interceptado.txt	ROT13 substitution	H4U{r0t_c1ph3r_br0k3n}
A2	A2_oracle.py, A2_admin_cipher.b64	AES-ECB oracle attack	H4U{4es_3cb_b10ck_4tt4ck}
A3	A3_documento_clasificado.txt	Vigenère + Kasiski (key: NEXATECH)	H4U{v1g3n3r3_k4s1sk1_4tt4ck}
A4	A4_mensaje_xor.hex	XOR repeating key (crib-dragging)	H4U{x0r_r3p34t1ng_k3y}
A5	A5_rsa_challenge.txt	RSA small exponent ($e=3$), cube root via gmpy2.iroot	H4U{rsa_sm4ll_3xp0n3nt}

Table 10.3: ServerA cryptography challenges (A5 is a key flag — required for ServerC access)

Required Python packages on ServerA: `pycryptodome 3.23.0`, `gmpy2 2.1.5`.

ServerB — Steganography Module

All challenge files reside in `/srv/public/uploads/` on ServerB.

ID	File(s)	Technique	Flag
B1	B1_oficina_novacorp.png	EXIF field Artist	H4U{3x1f_m3t4d4t4_h1dd3n}
B2	B2_network_diagram.png	Data appended after PNG IEND chunk	H4U{d4t4_4ft3r_1end_chunk}
B3	B3_documento_escaneado.png	LSB encoding in red channel	H4U{lsb_st3g4n0gr4phy_r}
B4	B4_novacorp_logo.png	LSB encoding in alpha channel (RGBA)	H4U{4lph4_ch4nn3l_h1dd3n}
B5	B5_camara_seguridad.jpg	Steghide JPEG (passphrase: novacorp)	H4U{st3gh1d3_p4ssw0rd_cr4ck}

Table 10.4: ServerB steganography challenges (B5 is a key flag — required for ServerC access)

Required packages on ServerB: `exiftool`, `steghide`, `python3-pil`.

ServerC — Advanced Mix Module

All challenge files reside in `/home/devops/retos/` on ServerC. Access to ServerC is gated on obtaining both A5 and B5 flags (see password derivation above).

ID	File(s)	Technique	Flag
C1	C1_backup_tapes.jpg	EXIF hint → steghide → ZIP multilayer	H4U{mult1l4y3r_st3g0_4es}
C2	C2_backup_log.b64	Onion encoding: <code>b64(bz2(b64(gz(flag))))</code>	H4U{0n10n_l4y3rs_st3g0}
C3	C3_api_token.txt, C3_verificador.py	SHA-1 length extension attack (hlexend)	H4U{l3ngth_3xt3ns10n_sh4}
C4	C4_ecdsa_signatures.txt, C4_verificador.py	ECDSA nonce reuse → secp256k1 private key recovery	H4U{3cc_n0nc3_r3us3_pwn3d}

Table 10.5: ServerC advanced challenges (C4 is the final boss flag)

Required packages on ServerC: `piexif`, `pillow`, `steghide`, `hlexend` (installed from GitHub), Python 3 standard library (`hashlib`, `struct`, `zlib`, `bz2`).

Flag Summary and Difficulty Map

ID	Difficulty	Flag value	Module
A1	Easy	H4U{r0t_c1ph3r_br0k3n}	Cryptography
A2	Easy-Medium	H4U{4es_3cb_b10ck_4tt4ck}	Cryptography
A3	Medium	H4U{v1g3n3r3_k4s1sk1_4tt4ck}	Cryptography
A4	Medium	H4U{x0r_r3p34t1ng_k3y}	Cryptography
A5	Medium-Hard	H4U{rsa_sm4ll_3xp0n3nt}	Cryptography
B1	Easy	H4U{3x1f_m3t4d4t4_h1dd3n}	Steganography
B2	Easy-Medium	H4U{d4t4_4ft3r_1end_chunk}	Steganography
B3	Medium	H4U{lsb_st3g4n0gr4phy_r}	Steganography
B4	Medium	H4U{4lph4_ch4nn3l_h1dd3n}	Steganography
B5	Medium	H4U{st3gh1d3_p4ssw0rd_cr4ck}	Steganography
C1	Hard	H4U{mult1l4y3r_st3g0_4es}	Advanced Mix
C2	Hard	H4U{0n10n_l4y3rs_st3g0}	Advanced Mix
C3	Hard	H4U{l3ngth_3xt3ns10n_sh4}	Advanced Mix
C4	Hard	H4U{3cc_n0nc3_r3us3_pwn3d}	Advanced Mix

Table 10.6: Lab 4 complete flag list with difficulty ratings

Lab Startup Checklist

1. Start all nodes from the EVE-NG web interface in order: pfSense → VyOS → ServerA → ServerB → ServerC → Defender.

2. Verify VyOS routing between zones:

```

1 # From VyOS console
2 show ip route
3 # Expected: 10.10.1.0/24, 10.10.2.0/24,
4 #           10.10.3.0/24, 10.10.4.0/24 all connected

```

3. Confirm SSH access to all server nodes:

```

1 ssh admin@10.10.1.10    # ServerA — N3xaTech!
2 ssh sysop@10.10.2.10    # ServerB — S3cure24!
3 ssh devops@10.10.3.10   # ServerC — fa681b59855d

```

4. Verify all challenge files are present on each server:

```

1 # ServerA
2 ls /home/admin/mensajes/
3 # A1_mensaje_interceptado.txt A2_oracle.py A2_admin_cipher.b64
4 # A3_documento_clasificado.txt A4_mensaje_xor.hex A5_rsa_challenge.txt

```

```

5
6 # ServerB
7 ls /srv/public/uploads/
8 # B1_oficina_novacorp.png B2_network_diagram.png
9 # B3_documento_escaneado.png B4_novacorp_Logo.png
   B5_camara_seguridad.jpg
10
11 # ServerC
12 ls /home/devops/retos/
13 # C1_backup_tapes.jpg C2_backup_Log.b64
14 # C3_api_token.txt C3_verificador.py
15 # C4_ecdsa_signatures.txt C4_verificador.py

```

5. Verify Python dependencies on each server:

```

1 # ServerA
2 python3 -c "from Crypto.Cipher import AES; import gmpy2; print('OK')"
3 # ServerB
4 python3 -c "from PIL import Image; print('OK')" && which exiftool
   steghide
5 # ServerC
6 python3 -c "import piexif, hlexend; from PIL import Image; print('OK
   ')"

```

Lab Reset Procedure

To restore the lab to its initial state after a student session, revert each server node to its pre-challenge EVE-NG snapshot. The challenge files can alternatively be re-deployed using the provisioning scripts at `src/eve-ng/configs/nodes/Lab4/` in the project repository:

```

1 # Re-stage ServerA challenges
2 bash src/eve-ng/configs/nodes/Lab4/serverA/deploy.sh
3 # Re-stage ServerB challenges
4 bash src/eve-ng/configs/nodes/Lab4/serverB/deploy.sh
5 # Re-stage ServerC challenges
6 bash src/eve-ng/configs/nodes/Lab4/serverC/deploy.sh

```